DISTRIBUTED SYSTEMS & CONCURRENCY
Course Code: CS-401 | Lecture Unit 1

1. Introduction to Distributed Systems
A distributed system is a collection of autonomous computing entities (nodes) that communicate over a network to coordinate actions and achieve common computational goals.

Key Characteristics:
- Concurrency of components
- Lack of a global shared clock
- Independent failure modes of individual components

2. The CAP Theorem
Formulated by Eric Brewer, the CAP theorem states that any distributed data store can only simultaneously provide at most two of the following three guarantees:
1. Consistency (C): Every read receives the most recent write or an error.
2. Availability (A): Every non-failing node returns a non-error response.
3. Partition Tolerance (P): The system continues to operate despite arbitrary message loss.

UNIT 2: CONSENSUS AND RAFT ALGORITHM

1. What is Distributed Consensus?
Consensus involves multiple server processes agreeing on values or state transitions despite unreliable networks and potential node crashes.

2. The Raft Consensus Algorithm
Designed by Ongaro and Ousterhout at Stanford (2014) for understandability.
Raft decomposes consensus into three independent sub-problems:
- Leader Election: When an existing leader fails, a new leader must be chosen.
- Log Replication: The leader accepts log entries from clients and forces followers to replicate them.
- Safety: If any server has applied a particular log entry to its state machine, no other server may apply a different log entry for the same log index.

Election Timeouts:
Followers randomize their election timeouts between 150ms and 300ms to split votes and ensure a swift single leader is elected.

UNIT 3: TWO-PHASE COMMIT (2PC) & TRANSACTIONS

1. Distributed Atomic Transactions
A distributed transaction involves operations across multiple independent databases.
Atomic commitment requires that all participating nodes either commit the transaction together
or abort the transaction cleanly (All-or-Nothing property).

2. Two-Phase Commit Protocol Steps:
Phase 1: Prepare Phase
- The Coordinator sends a PREPARE query to all Cohorts.
- Cohorts execute the transaction locally up to the commit point, writing changes to write-ahead logs (WAL).
- Cohorts reply VOTE_COMMIT or VOTE_ABORT.

Phase 2: Commit Phase
- If ALL cohorts vote VOTE_COMMIT, Coordinator broadcasts GLOBAL_COMMIT.
- If ANY cohort votes VOTE_ABORT (or times out), Coordinator broadcasts GLOBAL_ABORT.
- Cohorts finalize changes and send ACK.

UNIT 4: VECTOR CLOCKS AND FAULT TOLERANCE

1. Lamport Timestamps & Vector Clocks
Because physical clocks drift, distributed systems use logical clocks.
A vector clock for a system of N processes is an array of N logical clocks.
Process i increments V[i] on internal events.
On message receive from process j with timestamp W:
  V[k] = max(V[k], W[k]) for all k, and V[i] = V[i] + 1.

2. Byzantine Fault Tolerance (BFT)
A Byzantine fault represents arbitrary failure where components may not merely crash, but can transmit contradictory or malicious information to different parts of the cluster.
Practical Byzantine Fault Tolerance (PBFT) tolerates up to f faulty nodes in a system of 3f + 1 total nodes.